

UNIVERZA V LJUBLJANI
FAKULTETA ZA RAČUNALNIŠTVO IN INFORMATIKO

Nik Katanović Leban

**Spletna aplikacija za prodajo in
evalvacijo avtomobilov**

DIPLOMSKO DELO

VISOKOŠOLSKI STROKOVNI ŠTUDIJSKI PROGRAM
PRVE STOPNJE

MENTOR: viš. pred. dr. Igor Rožanc

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani <http://www.gnu.org/licenses/>.

Besedilo je oblikovano z urejevalnikom besedil $\text{\LaTeX}$.

Kandidat: Nik Katanović Leban

Naslov: Spletna aplikacija za prodajo in evalvacijo avtomobilov

Vrsta naloge: Diplomski naloga na visokošolskem programu prve stopnje
Računalništvo in informatika

Mentor: viš. pred. dr. Igor Rožanc

Opis:

Besedilo teme diplomskega dela študent prepíše iz študijskega informacijskega sistema, kamor ga je vnesel mentor. V nekaj stavkih bo opisal, kaj pričakuje od kandidatovega diplomskega dela. Kaj so cilji, kakšne metode naj uporabi, morda bo zapisal tudi ključno literaturo.

Title: Web application for car sales and evaluation

Description:

The student shall transcribe the text of the diploma thesis topic from the student information system, where it was entered by the mentor. In a few sentences, they will describe what is expected of the candidate's thesis. What the objectives are, what methods should be used, and perhaps they will also list the key literature.

Rad bi se zahvalil svojemu mentorju Igorju Rožancu, za vso aktivno pomoč in svetovanje pri razvoju te diplomske naloge. Zahvalil bi se tudi vsem sodelavcem iz podjetja CargoX, ki so z mano delili svoje znanje in izkušnje, ter mi omogočili, da sem se lahko lotil tako zahtevnega projekta. Posebna zahvala gre moji družini, ki me je ves čas podpirala in mi stala ob strani.

Kazalo

Povzetek

Abstract

1	Uvod	1
1.1	Opis arhitekture sistema	2
2	Pregled področja in sorodnih del	5
3	Tematsko ozadje	9
3.1	Spletne aplikacije	9
3.2	Relacijske podatkovne baze	10
3.3	Mikrostoritvena arhitektura	11
3.4	Zajem podatkov iz spletnih virov	12
4	Metodologija	15
4.1	Načrtovanje podatkovne baze	16
4.2	Arhitektura zalednega sistema	19
4.3	Arhitektura uporabniškega vmesnika	23
4.4	Evalvacijski sistem	27
4.5	Infrastruktura in avtomatizacija	33
5	Sklep	39

Literatura	43
Članki v revijah	43
Članki v zbornikih konferenc	45
Ostala literatura	46
Dodatek	51

Povzetek

Naslov: Spletna aplikacija za prodajo in evalvacijo avtomobilov

Avtor: Nik Katanović Leban

Diplomsko delo obravnava problem določanja primerne tržne vrednosti rabljenega avtomobila. Pri prodaji ali nakupu vozila uporabniki pogosto nimajo zadostnega pregleda nad trenutnim stanjem na trgu, zato težko ocenijo, ali je določena cena primerna. Posledično lahko vozilo kupijo po previsoki ceni ali ga prodajo pod njegovo dejansko tržno vrednostjo. Za reševanje tega problema sem razvil spletno aplikacijo za objavo in pregled avtomobilskih oglasov, ki uporabnikom omogoča enostavno in varno prodajo ter nakup vozil. Poleg tega je bil implementiran sistem za avtomatsko ocenjevanje vrednosti vozil. Sistem temelji na zbiranju podatkov iz zunanjih virov, njihovi normalizaciji in primerjavi vozila s podobnimi vozili na trgu. Pri oceni vrednosti se upoštevajo ključne lastnosti vozila, kot so znamka, model, letnik in prevoženi kilometri. Za pridobivanje podatkov so bili uporabljeni javno dostopni podatkovni viri ter postopki samodejnega zajema podatkov s spletnih strani za prodajo vozil. Zbrane podatke sistem obdela in uporabi za izračun ocenjene tržne vrednosti vozila. Rezultat diplomskega dela je delujoča spletna aplikacija s podporo za avtomatsko vrednotenje vozil, ki uporabnikom pomaga pri sprejemanju bolj informiranih odločitev pri nakupu ali prodaji avtomobila.

Ključne besede: spletna aplikacija, avtomobili, vrednotenje, React, Express.js, Node.js, PostgreSQL, REST API, Docker, GitHub Actions, TypeScript, Python, OAuth2, CI/CD.

Abstract

Title: Web application for car sales and evaluation

Author: Nik Katanović Leban

This work addresses the problem of determining the appropriate market value of a used vehicle. When buying or selling a car, users often lack sufficient insight into current market conditions, making it difficult to assess whether a listed price is fair. As a result, vehicles may be purchased at inflated prices or sold below their actual market value. To address this problem, a web application for publishing and browsing vehicle listings was developed, enabling users to buy and sell vehicles in a simple and secure manner. In addition, an automated vehicle valuation system was implemented. The system is based on collecting data from external sources, normalizing the acquired data, and comparing a vehicle with similar vehicles currently available on the market. The valuation process takes into account key vehicle characteristics, including brand, model, year of first registration, and mileage. The required data was obtained from publicly available datasets and through automated data collection from vehicle marketplace websites. The collected data is processed and used to estimate the market value of a vehicle. The result of this thesis is a functional web application with integrated vehicle valuation support that helps users make more informed decisions when buying or selling used vehicles.

Keywords: web application, automobiles, evaluation, React, Express.js, Node.js, PostgreSQL, REST API, Docker, GitHub Actions, TypeScript, Python, OAuth2, CI/CD.

Poglavje 1

Uvod

Nakup in prodaja rabljenih avtomobilov predstavljata pomemben del avtomobilskega trga. Pri tem se kupci in prodajalci pogosto srečujejo s težavo določanja primerne tržne vrednosti vozila, saj nimajo zadostnega pregleda nad trenutno ponudbo in gibanjem cen na trgu. Asimetrija informacij med udeleženci je še posebej izrazita na trgu rabljenih vozil, kjer je vsako vozilo unikatno glede na svoje stanje, opremo in zgodovino uporabe [19]. Najpogostejši pristop k oceni vrednosti je zato primerjava z obstoječimi oglasi podobnih vozil, torej t.i. pristop primerljivih prodaj (angl. *comparable sales approach*) [22], vendar samostojno izvajanje takšne primerjave ni vedno zanesljivo. Oglasi lahko vsebujejo nerealno visoke ali nizke cene, uporabniki pa pogosto nimajo dovolj znanja in izkušenj, da bi glede na te podatke lahko ocenili dejansko vrednost vozila. Posledično lahko vozilo prodajo pod njegovo tržno vrednostjo ali ga kupijo po previsoki ceni.

Na trgu že obstajajo rešitve za ocenjevanje vrednosti vozil, vendar so mnoge plačljive, prilagojene tujim trgov ali pa temeljijo na zastarelih podatkih, zaradi česar njihove ocene pogosto ne odražajo dejanskega stanja na slovenskem trgu rabljenih vozil. Podroben pregled in primerjava obstoječih rešitev sta predstavljena v poglavju 2 (razpredelnica 2.1).

Cilj diplomskega dela je razvoj spletne aplikacije za objavo in pregled avtomobilskih oglasov ter razvoj sistema za samodejno ocenjevanje vrednosti vozil.

Sistem temelji na zbiranju podatkov iz zunanjih virov, njihovi normalizaciji in analizi primerljivih vozil na trgu [27]. Pri določanju ocenjene vrednosti se upoštevajo ključne lastnosti vozila, kot so znamka, model, letnik prve registracije in število prevoženih kilometrov. Dolgoročni cilj sistema je privabiti zadostno število uporabnikov, ki na platformi objavljajo lastne avtomobilske oglase, tako da bo potreba po podatkih iz zunanjih virov postopno upadla, tržno vrednost vozil pa bo mogoče ocenjevati na podlagi oglasov, zbranih znotraj platforme same.

Hipoteza diplomskega dela je, da je mogoče s pomočjo zbranih tržnih podatkov in primerjalne analize podobnih vozil uporabniku ponuditi uporabno in dovolj natančno oceno tržne vrednosti vozila. Dodatni cilj je preveriti, ali integracija takšnega sistema v spletno aplikacijo izboljša uporabniško izkušnjo pri nakupu in prodaji vozil.

Preostanek diplomskega dela je organiziran na naslednji način. Poglavje 2 podaja pregled obstoječih spletnih tržnic in specializiranih storitev za vrednotenje vozil ter jih primerja s predlagano rešitvijo. Poglavje 3 predstavi teoretična izhodišča in tehnologije, na katerih temelji razvita rešitev, med drugim arhitekturo odjemalec–strežnik, relacijske podatkovne baze, mikrostoritveno arhitekturo ter postopke zajemanja podatkov s spletnih virov. Poglavje 4 opiše metodologijo dela: načrtovanje podatkovne baze in arhitekture sistema, implementacijo zalednega in odjemalskega dela aplikacije, delovanje algoritma za vrednotenje vozil ter infrastrukturo in avtomatizacijo namestitve. Diplomsko delo se zaključi s poglavjem 5, v katerem so povzeti doseženi rezultati, kritična presoja omejitev rešitve in nakaže smeri za nadaljnje izboljšave.

1.1 Opis arhitekture sistema

Spletna aplikacija je sestavljena iz uporabniškega vmesnika (odjemalec), strežniškega dela aplikacije (zaledni sistem) in ločenega sistema za vrednotenje

vozil. Uporabniški vmesnik je razvit z uporabo knjižnice React in programskega jezika TypeScript v okolju Node.js različice 24. Zaledni sistem je razvit z uporabo ogrodja Express v programskem jeziku JavaScript v okolju Node.js različice 24. Sistem za vrednotenje vozil je implementiran kot ločena storitev v programskem jeziku Python različice 3.11. Za izvajanje posameznih komponent je uporabljena platforma Docker.

Docker kontejnerji:

- Zaledni sistem
- Postgres:16
- redis:7
- mailhog
- Evalvacijski sistem

Razvoj aplikacije je potekal v razvojnem okolju Visual Studio Code. Za avtomatizacijo preverjanja kode, izvajanja testov ter gradnje vsebnikov Docker je bil uporabljen sistem GitHub Actions, medtem ko so se periodična opravila izvajala s pomočjo časovno načrtovanih opravil (cron). Za gostovanje aplikacije je bil uporabljen virtualni strežnik ponudnika Hetzner. Namestitev in upravljanje aplikacije na strežniku je bilo izvedeno s pomočjo platforme Dokploy.

Tabela 1.1: Pregled tehnologij, uporabljenih v posameznih komponentah sistema.

Komponenta	Uporabljene tehnologije
Uporabniški vmesnik	React, TypeScript, Vite, Zustand, Axios
Zaledni sistem	Node.js 24, Express.js, JavaScript, Sequelize (ORM), JWT, bcrypt, BullMQ, Redis
Evalvacijski sistem	Python 3.11, FastAPI, Alembic, RapidFuzz
Podatkovna baza	PostgreSQL 16
Infrastruktura in avtomatizacija	Docker, GitHub Actions, cron, Hetzner (VPS), Dokploy, MailHog

Poglavje 2

Pregled področja in sorodnih del

Na področju prodaje rabljenih vozil obstaja več spletnih aplikacij, ki uporabnikom omogočajo objavo in pregled avtomobilskih oglasov. Med najbolj znanimi evropskimi ponudniki sta mobile.de in AutoScout24, ki združujeta veliko število oglasov iz različnih držav ter uporabnikom omogočata filtriranje vozil po številnih kriterijih. Za slovenski trg je najbolj razširjena platforma Avto.net, ki predstavlja osrednje mesto za objavo in iskanje rabljenih vozil. Nekatere izmed omenjenih platform uporabnikom ponujajo tudi osnovno oceno primernosti cene vozila. Mobile.de na primer pri posameznih oglasih prikazuje informacijo o tem, ali je cena vozila glede na primerljive oglase ugodna, primerna ali previsoka. Takšen pristop uporabnikom omogoča hitrejšo orientacijo na trgu, vendar praviloma ne ponuja možnosti samostojnega vrednotenja vozila na podlagi uporabniško vnesenih podatkov.

Poleg spletnih tržnic obstajajo tudi specializirane storitve za vrednotenje vozil. Med najbolj znanimi je Eurotax, ki se pogosto uporablja v avtomobilski industriji, zavarovalništvu in pri trgovcih z vozili. Njegove ocene temeljijo na obsežnih zbirkah podatkov o vozilih in tržnih cenah, vendar je dostop do storitve praviloma plačljiv. Zaradi poslovnega modela in omejenega dostopa podrobna analiza delovanja sistema ni mogoča. Na ameriškem trgu sta razširjeni storitvi Kelley Blue Book in CARFAX. Kelley Blue Book uporabnikom omogoča oceno vrednosti vozila na podlagi tehničnih podatkov in

tržnih analiz, medtem ko CARFAX poleg vrednotenja ponuja tudi vpogled v zgodovino vozila. Obe storitvi sta prilagojeni predvsem severnoameriškemu trgu in zato nista neposredno uporabni za slovenski trg rabljenih vozil.

Tabela 2.1: Primerjava izbranih obstoječih rešitev za prodajo in vrednotenje vozil.

Rešitev	Geografski trg	Prodaja oglasov	Ocena vrednosti vozila
mobile.de	Evropa (več držav)	Da	Osnoven indikator cene
AutoScout24	Evropa (več držav)	Da	Priporočilo cene, pri objavi
Avto.net	Slovenija	Da	Ne
Eurotax	Mednarodno (industrija, zavarovalništvo, trgovci)	Ne	Da, podrobna (plačljiv dostop)
Kelley Blue Book	ZDA	Ne	Da, podrobna (tehnični podatki, tržna analiza)
CARFAX	ZDA	Ne	Da (vrednotenje in zgodovina vozila)
Predlagana rešitev	Slovenija	Da	Da, primerjava s podobnimi vozili na trgu

Pregled obstoječih rešitev je pokazal, da večina platform bodisi omogoča prodajo vozil brez naprednega sistema za vrednotenje bodisi ponuja ločene plačljive storitve za ocenjevanje vrednosti vozil. Namen razvite rešitve je združiti obe funkcionalnosti v enotni spletni aplikaciji, ki uporabnikom omogoča objavo vozil, pregled trga in samodejno oceno vrednosti vozila na podlagi primerljivih vozil, dostopnih na trgu.

Poglavje 3

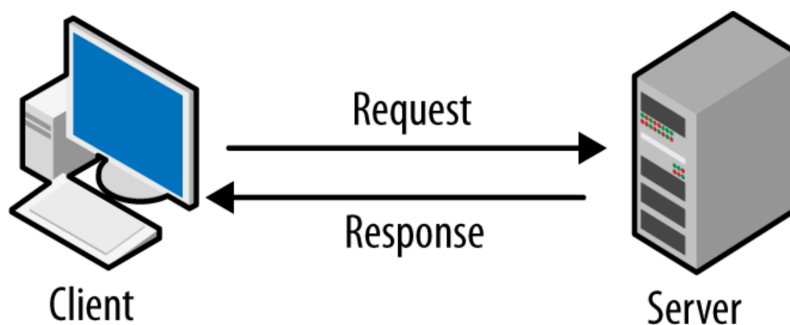
Tematsko ozadje

3.1 Spletne aplikacije

Spletne aplikacije sestavljata uporabniški vmesnik (odjemalec) in strežniški del aplikacije (zaledni sistem). Takšna arhitektura omogoča ločitev skrbi, kjer vsak del sistema prevzame svojo vlogo v celotni aplikaciji [42]. Uporabniški vmesnik omogoča prikaz podatkov ter interakcijo uporabnika z aplikacijo. Njegova glavna naloga je zagotavljanje preglednega, odzivnega in uporabniku prijaznega dostopa do funkcionalnosti sistema. Poleg prikaza podatkov skrbi tudi za zajem uporabniških vnosov, komunikacijo s strežniškim delom aplikacije ter lokalno shranjevanje določenih podatkov v spletnem brskalniku, kar prispeva k boljši odzivnosti in uporabniški izkušnji [15]. Sodobni uporabniški vmesniki so pogosto zasnovani na komponentni arhitekturi. Komponenta predstavlja samostojen in ponovno uporaben gradnik uporabniškega vmesnika, ki ga je mogoče vključiti na različnih mestih v aplikaciji. Takšen pristop izboljšuje modularnost sistema, poenostavlja vzdrževanje kode ter omogoča boljšo ponovno uporabo funkcionalnosti [30].

Strežniški del aplikacije je odgovoren za obdelavo zahtevkov, ki jih prejme od odjemalca, ter za vračanje ustreznih odgovorov. Poleg tega izvaja poslovno logiko sistema, upravlja z dostopnimi pravicami uporabnikov ter skrbi za avtentikacijo in avtorizacijo. Strežniški del je prav tako odgovoren za ko-

munikacijo s podatkovno bazo, kjer se hranijo trajni podatki aplikacije [14]. V sodobnih spletnih aplikacijah strežniški del pogosto omogoča dostop do podatkov preko programskih vmesnikov (vmesnik API). Eden izmed najpogostejše uporabljenih arhitekturnih pristopov je REST, ki temelji na brezstanovnem (angl. *stateless*) načinu komunikacije med sistemi in uporablja standardne HTTP metode za izvajanje operacij nad viri [14]. Poleg osnovne funkcionalnosti lahko strežniški del vključuje tudi dodatne mehanizme, kot so omejevanje števila zahtevkov (angl. *rate limiting*), predpomnjenje podatkov ter beleženje dogodkov, kar prispeva k večji varnosti, stabilnosti in zmogljivosti sistema [40].



Slika 3.1: Arhitektura spletne aplikacije tipa odjemalec–strežnik [9].

3.2 Relacijske podatkovne baze

Relacijske podatkovne baze predstavljajo enega izmed najpogostejše uporabljenih pristopov za shranjevanje podatkov v spletnih aplikacijah. Temeljijo na relacijskem modelu, pri katerem so podatki organizirani v tabele, ki predstavljajo posamezne entitete. Vsaka entiteta vsebuje attribute, s katerimi opisujemo njene lastnosti, posamezne vrstice pa predstavljajo konkretne primerke entitete. Relacijski model omogoča strukturirano in konsistentno shranjevanje podatkov ter učinkovito izvajanje poizvedb nad velikimi količinami podatkov [12]. Povezave med entitetami vzpostavimo s primarnimi in tujimi ključi. Primarni ključ enolično identificira posamezen zapis v tabeli, medtem

ko tuji ključ omogoča povezovanje podatkov med različnimi tabelami. Na ta način zmanjšamo podvajanje podatkov ter zagotovimo referenčno integriteto podatkovne baze [12].

Pri razvoju sistema smo izbrali PostgreSQL, ki predstavlja odprtokodni objektno-relacijski sistem za upravljanje podatkovnih baz. PostgreSQL temelji na standardu SQL in ga razširja z naprednimi funkcionalnostmi, ki omogočajo učinkovito delo tako s strukturiranimi kot tudi s polstrukturiranimi podatki [47]. Med pomembnejšimi lastnostmi sistema PostgreSQL so podpora za transakcije ACID, napredne poizvedbe SQL, replikacija podatkov, mehanizmi za zagotavljanje integritete podatkov ter podpora za podatkovni format JSON oziroma JSONB. Zaradi teh funkcionalnosti je PostgreSQL primeren za razvoj sodobnih spletnih aplikacij, ki zahtevajo visoko zmogljivost, zanesljivost in možnost nadaljnje razširljivosti sistema [47, 46].

3.3 Mikrostoritvena arhitektura

Mikrostoritvena arhitektura je arhitekturni vzorec, pri katerem se programski sistem razdeli na množico manjših, med seboj neodvisnih storitev, namesto da bi bil zgrajen kot ena tesno povezana enota, znana kot monolitna arhitektura [11]. Vsaka mikrostoritev implementira natančno določeno poslovno funkcionalnost in je razvita, nameščena ter skalirana neodvisno od preostalih storitev v sistemu. Komunikacija med posameznimi mikrostoritvami praviloma poteka prek dobro definiranih vmesnikov, kot so vmesniki API, sporočilne vrste ali drugi mehanizmi za izmenjavo podatkov [34].

Razdelitev sistema na več manjših, neodvisnih komponent prinaša več prednosti v primerjavi z monolitno arhitekturo [34]. Posamezne mikrostoritve je lažje testirati, saj imajo omejen in jasno določen obseg delovanja. Prav tako je olajšano odkrivanje in odpravljanje napak, saj je napako mogoče hitreje lokalizirati znotraj posamezne storitve, namesto da bi jo iskali znotraj celotnega sistema. Neodvisnost storitev dodatno omogoča lažje vzdrževanje, saj je mogoče posodobiti ali zamenjati eno mikrostoritev, ne da bi to vplivalo

na delovanje ostalih mikrostoritev v sistemu, pod pogojem, da vmesnik za komunikacijo med njimi ostane nespremenjen [11].

3.4 Zajem podatkov iz spletnih virov

Zajem podatkov iz spletnih virov predstavlja temeljni korak pri vrednotenju rabljenih vozil na trgu. Kakovost, obseg in svežost zbranih podatkov neposredno vplivajo na zanesljivost modelov za ocenjevanje cen. [27]. V splošnem ločimo tri pristope za pridobivanje podatkov iz spleta: izkoriščanje javno dostopnih podatkovnih zbirk, samodejni zajem podatkov s spletnih strani (ang. *web scraping*) ter dostop prek vmesnikov API [27].

3.4.1 Javno dostopne podatkovne zbirke

Javno dostopne podatkovne zbirke predstavljajo enega najprimernejših virov za zbiranje strukturiranih podatkov, saj so podatki v njih že organizirani, kategorizirani in pripravljeni za neposredno analitično obdelavo [23]. Primeri takšnih zbirk so odprti vladni portali, kot sta Statistični urad Republike Slovenije (SURS) [44] in Eurostat [13], akademski repozitoriji ter portali prometnih agencij. Slabost tega pristopa je omejena razpoložljivost aktualnih tržnih podatkov. Javne zbirke navadno ne vsebujejo oglasov z aktualnimi prodajnimi cenami vozil, ki so nujno potrebni za ocenjevanje tržne vrednosti [19]. Za tuje trge sicer obstajajo javno dostopne zbirke oglasov, pridobljenih z oglasnih portalov, npr. nemški trg (eBay Kleinanzeigen) [36], vendar za slovenski trg takšna javna zbirka ne obstaja.

3.4.2 Samodejni zajem podatkov s spletnih strani

Samodejni zajem podatkov s spletnih strani je postopek avtomatiziranega pridobivanja strukturiranih informacij iz spletnih strani z razčlenjevanjem njihove vsebine [27]. Ta pristop se pogosto uporablja pri zbiranju oglasov za prodajo rabljenih vozil, saj je na voljo veliko portalov, ki vsebujejo obsežne

zbirke takšnih oglasov. Iz vsakega oglasa je mogoče pridobiti strukturirane attribute vozila, kot so znamka, model, letnik izdelave, prevoženi kilometri, vrsta goriva, tip menjalnika in oglašena prodajna cena [19]. Zajem poteka z orodji za avtomatizirano brskanje po spletnih straneh, ki simulirajo obisk uporabnika, ter z razčlenjevanjem strukture dokumenta HTML za izluščitev relevantnih podatkovnih polj [27].

3.4.3 Razčlenjevanje vsebine HTML

Spletna stran je v osnovi dokument v jeziku HTML (ang. *HyperText Markup Language*), ki je organiziran v drevesno strukturo DOM (ang. *Document Object Model*) [35]. Vsak element dokumenta, kot so naslovi, odstavki in podatkovne celice, je v tej strukturi predstavljen kot vozlišče, ki ima lahko starševska in podrejena vozlišča. Orodja za razčlenjevanje (ang. *parsers*), kot sta knjižnici `BeautifulSoup` in `lxml` v jeziku Python, to strukturo pretvorijo v obliko, primerno za programsko obdelavo.

Za izluščevanje podatkovnih elementov iz razčlenjene strukture se v praksi najpogosteje uporabljata dve vrsti izbirnikov: izbirniki CSS in izbirniki XPath [8]. Izbirniki CSS so bili prvotno zasnovani za oblikovanje spletnih strani, a jih pri zajemu podatkov preusmerimo v poizvedovanje po strukturi DOM [35]. Izbirniki XPath (ang. *XML Path Language*) pa so namensko poizvedovalni jezik za navigacijo po drevesnih strukturah XML in HTML, ki poleg preprostega iskanja po oznakah in razredih podpira tudi premikanje navzgor po drevesu, iskanje po vsebini besedilnih vozlišč ter vgrajene funkcije, kot sta `contains()` in `position()` [8]. V kontekstu zajemanja podatkov iz avtomobilskih oglasnih portalov to pomeni, da z izbirniki iz razčlenjenega dokumenta izluščimo posamezna podatkovna polja oglasa, kot so naziv vozila, cena, letnik, prevoženi kilometri in vrsta goriva.

Poglavje 4

Metodologija

Cilj diplomske naloge je bila izdelava spletne aplikacije, v kateri bodo uporabniki lahko objavljali oglase svojih avtomobilov. Pred začetkom razvoja smo morali izbrati ustrezne tehnologije, kar smo storili na podlagi lastnih izkušenj in predznanja ter pregleda različnih forumov in strokovnih člankov. Ko smo zaključili razvoj spletne aplikacije, smo se lotili razvoja sistema za vrednotenje avtomobilov, napisanega v programskem jeziku Python. Python smo izbrali, ker je zelo primeren za obdelavo podatkov in razpolaga z bogatim ekosistemom knjižnic, namenjenih analizi podatkov.

Pri izbiri orodij za razvoj uporabniškega vmesnika smo se odločili za ogrodje React. Odločitev je deloma temeljila na želji po pridobitvi novih znanj v okviru diplomskega dela, saj z Reactom pred tem nismo imeli izkušenj. React je eno izmed najpopularnejših ogrodij za razvoj uporabniških vmesnikov z eno največjih skupnosti, kar zagotavlja kakovostno dokumentacijo in dolgoročno podporo. Skupaj z Reactom smo uporabili jezik TypeScript, ki je nadgradnja jezika JavaScript [31]. TypeScript razvijalce prisili k eksplicitni deklaraciji tipov spremenljivk, kar bistveno zmanjša možnost napak med razvojem.

Za razvoj zalednega sistema smo izbrali ogrodje Express.js, zgrajeno na platformi Node.js, ki omogoča hitro in enostavno izdelavo spletnih aplikacij. Odločitev je temeljila na predhodnih izkušnjah z ogrođjem ter na njegovi eno-

stavni integraciji z vmesniki REST API. Zaledni sistem smo razvili v jeziku JavaScript, kar se je naknadno izkazalo za manj optimalno odločitev. Jezik TypeScript bi z dodatno varnostjo tipov prispeval k boljši zanesljivosti in vzdržljivosti kode.

4.1 Načrtovanje podatkovne baze

4.1.1 Načrtovanje podatkovne baze aplikacije

Pri načrtovanju podatkovne baze smo morali natančno premisliti, kako shranjujemo oglase avtomobilov, saj je to ključnega pomena za učinkovito delovanje odločitvenega modela. Previsoka stopnja normalizacije bi upočasnila poizvedbe, prenizka pa bi vodila do podvajanja podatkov in posledičnih neskladnosti v bazi [12].

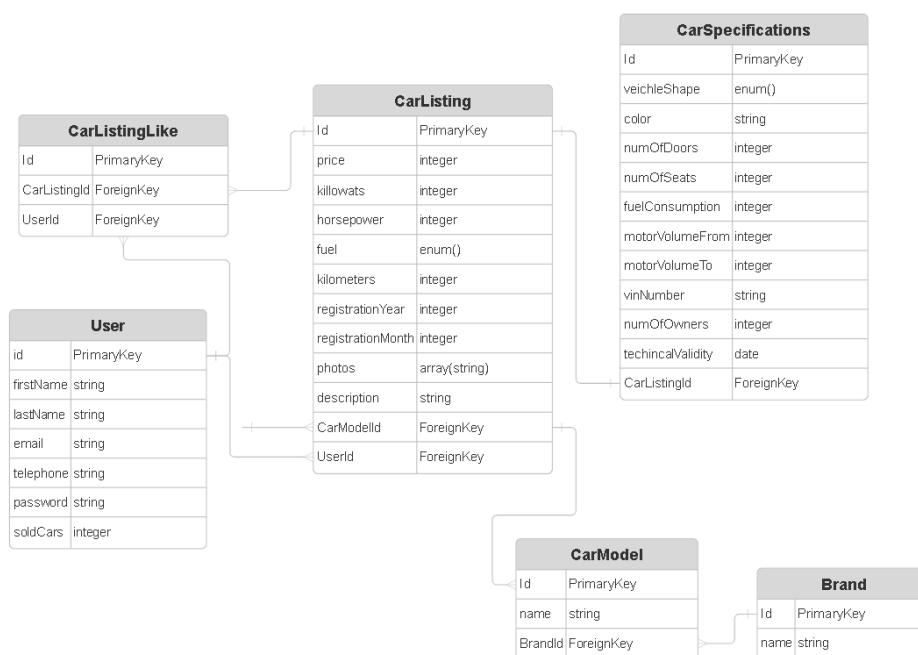
Entiteti `Brand` in `CarModel` sta izločeni v ločeni tabeli, ker med njima obstaja hierarhično razmerje ena-proti-mnogo: ena znamka (`Brand`) ima lahko poljubno število modelov (`CarModel`), vsak model pa pripada natanko eni znamki. Shranjevanje znamk in modelov kot vrednosti tipa `enum` ali prostega niza bi bilo neprimerno iz dveh razlogov. Prvič, število možnih vrednosti je preveliko za statično definicijo z `enum`. Drugič, prosti niz bi omogočal podvojene ali nekonsistentne vnose – na primer „Audi A3“ in „audi a3 Sportback“ bi bila obravnavana kot različni entiteti, čeprav gre za isti model. Z relacijsko vezavo prek tujih ključev je zagotovljen referenčni nadzor nad vnesenimi vrednostmi ter enotnost podatkov.

Entiteta `CarSpecifications` je namerno ločena od entitete `CarListing`, ker vsebuje tehnične podrobnosti vozila, ki niso relevantne pri vsakem poizvedovanju. Pri prikazu seznama oglasov (npr. v podatkovni tabeli) zadostujejo le osnovni podatki iz entitete `CarListing` (cena, prevoženi kilometri, letnik ...), specifikacije pa se naložijo le ob ogledu posameznega oglasa. Ta ločitev zmanjša količino podatkov, ki se prenesejo iz podatkovne baze pri pogostih

poizvedbah, in s tem izboljša odzivnost aplikacije.

Entiteta `CarListingLike` predstavlja asociativno tabelo, ki realizira razmerje mnogo-proti-mnogo med uporabniki in oglasi: en uporabnik lahko označi kot všečkano poljubno število oglasov, en oglas pa je lahko všečkan pri poljubnem številu uporabnikov. Tabela vsebuje tuji ključ na entiteto `User` in tuji ključ na entiteto `CarListing`, kar zagotavlja referenčno integriteto in preprečuje podvojene vnose.

Entiteta `User` hrani standardne podatke o registriranih uporabnikih sistema.



Slika 4.1: ER-diagram glavnih entitet aplikacije.

4.1.2 Načrtovanje podatkovne baze odločitvenega modela

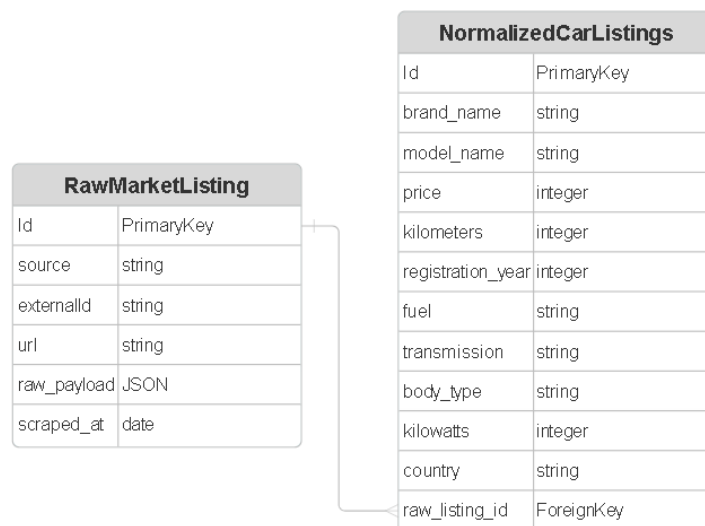
V okviru iste podatkovne baze smo dodali ločeno shemo **market**, ki vsebuje dve tabeli, namenjeni zbiranju in obdelavi tržnih podatkov za sistem vredno-

tenja avtomobilov.

Tabela `RawMarketListing` služi kot surov arhiv vseh oglasov, pridobljenih iz zunanjih virov – javno dostopnih zbirk podatkov, spletnih vmesnikov API in z zajemanjem spletnih strani. Tabela vsebuje naslednja polja: `source` identificira izvor podatkov (npr. ime portala ali vmesnika API), `externalId` je identifikator oglasa na izvornem sistemu, ki je določen statično v kodi in preprečuje podvojene vnose, `url` hrani neposredno povezavo do oglasa, `scraped_at` je časovni žig trenutka zajema podatkov, `raw_payload` pa vsebuje izvorni odgovor v obliki JSON brez kakršnekoli obdelave. Izvorni odgovor je namerno shranjen v celoti, kar omogoča kasnejšo ponovno obdelavo brez ponovnega zajema podatkov, če se normalizacijska logika spremeni.

Tabela `NormalizedCarListing` vsebuje obdelano in normalizirano obliko surovega oglasa, vezano na tabelo `RawMarketListing` prek tujega ključa `raw_listing_id`. Razmerje med tabelama je ena-proti-mnogo: vsak surovi oglas ima lahko več normaliziranih zapisov. Polja so poenotena in tipizirana – cena, prevoženi kilometri in letnik so shranjeni kot `integer`, znamka, model, vrsta goriva, tip menjalnika, oblika karoserije in država pa kot `string`. Takšna poenotena struktura omogoča neposredno primerjavo oglasov iz različnih virov pri izračunu tržne vrednosti vozila.

Ločitev na dve tabeli sledi vzorcu cevovoda surovih in normaliziranih podatkov (angl. *raw/normalized pipeline*) [26]: surovi podatki ostanejo nedotaknjeni kot revizijska sled in osnova za morebitno ponovno obdelavo, normalizirana oblika pa je tista, ki se uporablja v analitičnih poizvedbah.



Slika 4.2: ER-diagram entitet Evalvacijskega sistema.

4.2 Arhitektura zalednega sistema

Zaledni sistem smo oblikovali z modularno arhitekturo, ki omogoča jasno ločitev odgovornosti (separation of concerns), kar olajša branje, vzdrževanje in nadaljnjo razširitev kode [10]. Kot ogrodje za zaledni sistem smo izbrali Express.js, minimalistično ogrodje za Node.js, ki zagotavlja temeljne funkcionalnosti za usmerjanje zahtev in upravljanje vmesnika HTTP, hkrati pa razvojnikom omogoča svobodno izbiro specifičnih knjižnic glede na različne zahteve aplikacije [6].

4.2.1 Modul za poizvedbe v podatkovni bazi

Za implementacijo podatkovnih modelov in poizvedkov na podatkovno bazo smo uporabili knjižnico Sequelize. Sequelize je vmesnik za spreminjanje objektno-relacijskih preslikav (ORM – Object-Relational Mapping), ki omogoča preslikavo objektov iz programske kode v relacijske strukture podat-

kovne baze [7]. ORM pristop izboljšuje razvoj, saj omogoča programerjem, da delajo z objekti namesto neposrednih poizvedb SQL, kar zmanjša napake in poveča berljivost kode [7]. Prednosti ORM pristopa vključujejo tipsko varnost, avtomatično generiranje poizvedb za različne podatkovne baze ter intuitivnejše upravljanje relacij med entitetami.

Podatkovni modeli so v Sequelize definirani kot razredi, ki predstavljajo tabele v bazi podatkov [7]. Vsak model ima attribute, ki ustrezajo stolpcem v tabeli, ter metode za izvajanje temeljnih operacij s podatki (CRUD – Create, Read, Update, Delete). Ključno pri oblikovanju modelov je tudi postavitvev indeksov na poljih, ki se pogosto pojavljajo v pogojih poizvedb (npr. ID avtomobila, izvor oglasa ali časovni žigi). Dobro zasnovani indeksi so bistvenega pomena za zagotavljanje hitrih odgovorov poizvedb pri velikih količinah podatkov [18]. Primer definirane modelne strukture je prikazan v Kodi 8.

Izsek kode 1: Koda za vzpostavitev povezave z bazo podatkov.

```
1 import { Sequelize } from 'sequelize';
2
3 const sequelize = new Sequelize(process.env.DB_NAME, process.env.DB_USER,
4   ↪ process.env.DB_PASSWORD, {
5     host: process.env.DB_HOST,
6     port: process.env.DB_PORT,
7     dialect: 'postgres',
8     logging: false,
9   });
10 export default sequelize;
```

4.2.2 Avtentikacija in avtorizacija uporabnikov

Za avtentikacijo uporabnikov smo implementirali sistem, ki temelji na žetonih JWT (JSON Web Token), kar omogoča varen in enostaven prenos podatkov o uporabnikovi pristnosti [24]. Žeton JWT je strukturiran dokument, sestavljen iz treh delov: glave (header), ki vsebuje metapodatke o vrsti kodiranja;

podatkov (payload), ki nosijo informacije o uporabniku; in digitalnega podpisa, ki zagotavlja celovitost in pristnost žetona [24]. Postopek avtentikacije je razdeljen na dve fazi. V fazi registracije uporabnik vnese svoje geslo, ki ga pred shranitvijo v podatkovno bazo zgoščimo z algoritmom bcrypt [38]. Bcrypt je specializirana kriptografska funkcija, namenjena zgoščevanju gesel, ki vključuje samodejno upoštevanje (salting) in je odpornejša na napade [38]. V fazi prijave uporabnik pošlje svoje geslo in e-poštni naslov. Zaledni sistem pridobi shranjeno zgoščeno geslo iz baze in ga primerja z novim zgoščenim geslom s pomočjo funkcije bcrypt za primerjavo. Če se gesla ujemata, zaledni sistem generira žeton JWT z informacijami o uporabniku in ga pošlje nazaj na uporabniški vmesnik, kjer ga uporabnik shrani v brskalniku (največkrat v shrambo seje – localStorage).

Za zaščito poti vmesnika API smo implementirali middleware za preverjanje žetonov. Vmesnik je funkcija, ki se izvede pred vsakim zahtevkom in preveri, ali je uporabnik poslal veljaven žeton JWT v zahtevku. Dodatna varnostna plast je nastavljena s časovnimi omejitvami žetona, ki zagotavlja, da je žeton veljaven samo za določen čas. Po preteku roka veljavnosti se mora uporabnik ponovno avtentificirati in pridobiti nov žeton, kar zmanjšuje tveganje ob morebitnem razkritju starejših žetonov [43].

Izsek kode 2: Koda za prijavo uporabnika.

```
1 export const loginUserService = async ({ email, password }) => {
2   try {
3     if (!email || !password) throw new MissingUserDataError();
4
5     const user = await User.findOne({ where: { email } });
6
7     if (!user) throw new AppError('Invalid email or password', 401);
8
9     const isPasswordValid = await bcrypt.compare(password, user.password);
10
11    if (!isPasswordValid) throw new AppError('Invalid email or password',
12      ↪ 401);
```

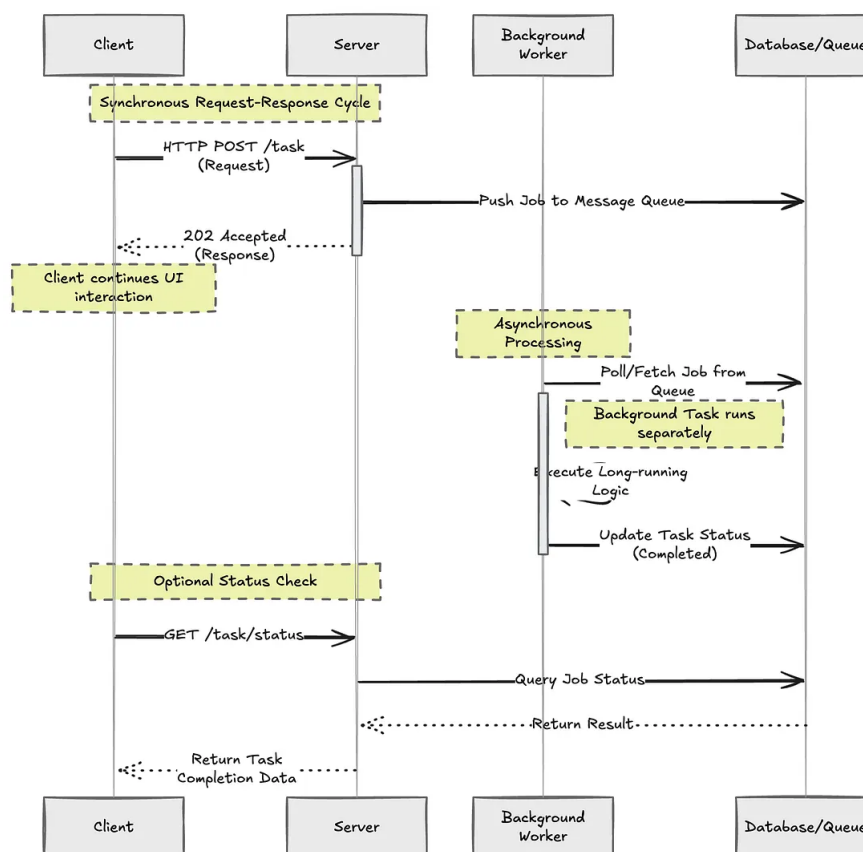
```
13   const token = jwt.sign({ id: user.id, email: user.email },  
    ↪   process.env.JWT_SECRET, {  
14     expiresIn: '7d',  
15   });  
16  
17   await emailQueue.add('login', { type: 'login', to: user.email,  
    ↪   userName: user.firstName });  
18  
19   return { user, token };  
20 } catch (error) {  
21   throw error;  
22 }  
23 };
```

4.2.3 Asinkrona obdelava podatkov (Celery)

Za naloge, ki zahtevajo obsežnejšo obdelavo podatkov ali imajo daljši čas izvajanja, smo implementirali asinkrono obdelavo z uporabo sistema čakalnih vrst nalog. Asinkroni pristop omogoča, da strežnik takoj odgovori na zahtevek, medtem ko se zahtevnejše operacije izvajajo v ozadju kot ločeni procesi. Na ta način se zmanjšuje čakalni čas uporabnika in izboljša splošna odzivnost aplikacije [26]. Značilne naloge, ki zahtevajo asinkrono obdelavo, vključujejo zajemanje podatkov iz zunanjih virov, normalizacijo in transformacijo zajetih podatkov ter pošiljanje e-poštnih obvestil uporabnikom [26]. Za upravljanje čakalnih vrst nalog v okolju Node.js smo izbrali knjižnico BullMQ, ki zagotavlja zanesljivo upravljanje asinhronih nalog z vgrajeno podporo za ponovne poskuse (angl. *retry*), časovne omejitve (angl. *timeout*) in obravnavo napak [5]. Sistem temelji na kombinaciji podatkovnega skladišča Redis in delavskih procesov. Redis v tej arhitekturi služi kot posrednik sporočil (angl. *message broker*), ki hrani stanje nalog in omogoča koordinacijo med posameznimi delavskimi procesi [41].

Konfiguracija sistema zahteva tri ključne korake. Najprej se vzpostavi povezava s strežnikom Redis, ki deluje kot posrednik med čakalno vrsto in delavci. Nato se opredelijo posamezne vrste nalog (angl. *job types*) z ustreznimi funkcijami za njihovo izvajanje. Na koncu se registrirajo delavski procesi, ki

poslušajo dodeljene vrste nalog, jih obdelujejo in ustrezno posodabljaajo stanje v skladišču Redis. S takšno zasnovo je zagotovljeno, da se dolgotrajne operacije ne izvajajo na glavni niti aplikacije, kar omogoča sočasno obdelavo in izboljšano razširljivost sistema.



Slika 4.3: Asinkrona obdelava podatkov z uporabo čakalnih vrst nalog in delavskih procesov [32].

4.3 Arhitektura uporabniškega vmesnika

Uporabniški vmesnik smo zasnovali iz več komponent, ki skupaj sestavljajo celoto. Pri tem smo upoštevali načela modularnosti, ki omogočajo ponovno uporabo komponent ter lažje vzdrževanje kode [37]. Vsako komponento smo oblikovali tako, da opravlja natančno določeno nalogo in prikazuje omejen del

vmesnika. S tem smo sledili načelu enotne odgovornosti, po katerem naj ima vsaka enota kode en sam razlog za spremembo [28]. Komponente smo namenoma ohranili majhne, saj majhen obseg naravno usmerja razvijalca k temu, da posamezna komponenta ostane osredotočena na eno samo funkcionalnost. Komponente si med seboj izmenjujejo podatke po hierarhiji nadrejenih in podrejenih komponent. Nadrejena komponenta posreduje podatke podrejenim komponentam prek lastnosti (angl. *props*), podrejena komponenta pa nadrejeni sporoča spremembe prek povratnih funkcij, ki jih nadrejena komponenta posreduje navzdol. Kot ogrodje za izgradnjo uporabniškega vmesnika smo izbrali React, ki je eno izmed najbolj razširjenih in uveljavljenih ogrodij za razvoj spletnih vmesnikov [29]. V naslednjih podpoglavjih podrobneje predstavimo posamezne dele te arhitekture.

4.3.1 Odjemalska shramba

Določene podatke si mora odjemalec zapomniti, kar pomeni, da jih mora shraniti na svoji strani. Prednost odjemalske shrambe je, da odjemalcu ni treba nenehno pošiljati novih zahtevkov na strežnik, kar občutno zmanjša njegovo obremenitev. Značilen primer uporabe je shranjevanje žetona JWT, ki ga uporabnik prejme ob prijavi in ga nato pri vsakem zahtevku pošilje v glavi [24]. Na ta način imamo podatke o prijavljenem uporabniku ves čas na voljo, dokler se ne odjavi ali dokler žeton JWT ne poteče.

Odjemalska shramba je uporabna tudi pri večkoračnih postopkih, saj si lahko zapomnimo, do katerega koraka je uporabnik prišel. Če uporabnik namerno ali nenamerno zapre stran, ostane njegovo napredovanje ohranjeno in lahko z delom nadaljuje. Pri shranjevanju podatkov na odjemalcu moramo biti pozorni, da hranimo le neobčutljive podatke ter da stanje pravilno posodabljammo ob vsaki spremembi. Spletni brskalniki za trajno hrambo podatkov na strani odjemalca ponujajo več mehanizmov, med drugim `localStorage` in `sessionStorage` [50]. V nasprotnem primeru se lahko stanje na odjemalčevi strani razlikuje od stanja na strežniku, kar vodi do težko odkrивljivih

napak.

Za odjemalsko shrambo smo uporabili knjižnico Zustand, ki je preprosta, kompaktna in dobro vzdrževana. Zustand temelji na minimalističnem pristopu k upravljanju stanja in za osnovno delovanje ne zahteva ovojnih komponent (angl. *provider*), kar ga loči od nekaterih uveljavljenih rešitev, kot je Redux [**zustand-docs**]. Stanje je organizirano v obliki t.i. shramb (angl. *store*), do katerih dostopamo prek preprostih kavljev (angl. *hooks*), pri čemer se ponovno izrišejo le tiste komponente, ki dejansko uporabljajo spremenjeni del stanja. Zaradi majhne velikosti in neodvisnosti od ogrodja React je knjižnica primerna tudi za projekte, pri katerih želimo ohraniti nizko zahtevnost in dobro zmogljivost.

4.3.2 Pisanje zahtevkov API

Za pošiljanje zahtevkov smo uporabili knjižnico Axios, ki je ena izmed najbolj razširjenih knjižnic za pošiljanje zahtevkov HTTP v okolju JavaScript [2]. Nad njo smo izdelali ovojnico `api`, ki iz okoljskih spremenljivk (angl. *environment variables*) prebere naslov strežnika in ga uporabi kot osnovni naslov URL za vse zahteve. Tak pristop nam omogoča, da osnovni naslov in skupne nastavitve določimo na enem mestu, s čimer se izognemo ponavljanju in poenostavimo morebitne kasnejše spremembe.

Izsek kode 3: Ovojnica Axios za ustvarjanje odjemalca API.

```
1 import axios from "axios";
2
3 const api = axios.create({
4   baseURL: import.meta.env.VITE_API_BASE_URL,
5   withCredentials: true,
6 });
7
8 export default api;
```

Ovojnico smo nato uporabili pri vsaki definiciji funkcije za pošiljanje zahtevka. Funkcija vrne polje `res.data`, ki vsebuje dejanski odgovor strežnika. Axios namreč vrne objekt s številnimi dodatnimi podatki, ki niso vedno potrebni, zato zaradi preglednosti kode vrnemo le vsebino odgovora [2]. Po potrebi lahko dostopamo tudi do statusne kode strežnika, glave odgovora in konfiguracije zahtevka.

Izsek kode 4: Primer definicije funkcije za pridobivanje znamk avtomobilov.

```
1 export const fetchCarBrands = async () => {  
2   const res = await api.get<{ brandNames: string[] }>("/api/cars/brands");  
3   return res.data;  
4 };
```

4.3.3 Vite kot razvojno in gradbeno orodje

Vsako izvorno kodo je treba pred prikazom v spletnem brskalniku ustrezno pripraviti oziroma zgraditi. To nalogo v našem projektu opravlja orodje Vite, ki omogoča hitro postavitev razvojnega okolja in gradnjo končne različice aplikacije [48]. Za razliko od starejših orodij za združevanje kode (angl. *bundler*), kot je Webpack, ki mora ob vsaki spremembi znova zgraditi večji del kode, Vite tega ne počne in je zato bistveno hitrejši.

Delovanje orodja Vite temelji na dveh ključnih načelih. Prvo je izraba native podpore modulom ECMAScript (angl. *ES modules*) v sodobnih brskalnikih. Ker sodobni brskalniki znajo neposredno razumeti module, uvožene v kodi, Vite med razvojem kode ne združuje, temveč posamezne module dostavi brskalniku po potrebi [33]. Drugo načelo je uporaba orodja esbuild za predhodno gradnjo odvisnosti JavaScript. Orodje esbuild je napisano v jeziku Go in je od 10 do 100-krat hitrejše od orodij za združevanje, napisanih v jeziku JavaScript [49].

Pomembna prednost orodja Vite je zmožnost sprotnega osveževanja modulov (angl. *Hot Module Replacement*), pri katerem se osveži le tisti del kode,

ki se je spremenil, brez izgube trenutnega stanja in podatkov aplikacije. Ta zmožnost je pri razvoju izjemno pomembna, saj občutno pospeši povratno zanko med spremembo in prikazom ter tako poveča produktivnost. Vite v naši arhitekturi torej opravlja vlogo razvojnega in gradbenega sloja.

4.4 Evalvacijski sistem

Evalvacijski sistem smo zasnovali kot samostojno storitev, ki smo jo razvili v programskem jeziku Python. Začeli smo z osnovnim jezikom in postopno dodajali pakete glede na potrebe. Za komunikacijo z zalednim sistemom smo potrebovali spletni strežnik, saj je zaledni sistem edini, ki ima dostop do evalvacijskega sistema. Za spletni strežnik in vmesnik API smo uporabili ogrodje FastAPI, ki omogoča enostavno izdelavo vmesnikov API v jeziku Python [39].

Izsek kode 5: Primer definicije vmesnika API z ogrođjem FastAPI

```
1 @app.get("/car-brands")
2 def get_car_brands():
3     return getCarBrands()
```

Podatkovno bazo smo ločili v samostojno shemo z lastnimi modeli, s čimer smo se izognili morebitni zmedi pri poimenovanju in strukturi. Za migracijo tabel in podatkov smo dodali paket Alembic, ki je zmogljivo orodje za upravljanje in posodabljanje podatkovnih baz [3]. Alembic sledi shemi podatkovne baze in samodejno ustvarja migracijske skripte, ki jih je mogoče verzionirati skupaj z izvorno kodo, kar zagotavlja ponovljivo in nadzorovano spreminjanje strukture podatkovne baze. Pri tem projektu smo se prvič srečali s tehnologijo cron. Cron je programska oprema, ki omogoča samodejno izvajanje ukazov ob vnaprej določenih časovnih intervalih na strežniku ali v drugem okolju [45]. Cron smo uporabili za zagon programov za samodejni zajem podatkov s spletnih strani za prodajo vozil. Ti programi se na strežniku izvajajo dnevno oziroma tedensko.

4.4.1 Algoritem za primerjavo vozil

Vse podatke, pridobljene z zajemom s spletnih strani ali z dostopom do javno dostopnih virov, smo morali najprej shraniti v nestrukturirani obliki, nato pa jih normalizirati. Za vsak spletni vir smo morali napisati ločen postopek normalizacije, saj morajo biti podatki, če jih želimo primerjati, normalizirani oziroma zapisani v enotni obliki. V nasprotnem primeru pri evalvaciji prihaja do diskrepanc. Za vsako vozilo smo shranili ceno, prevožene kilometre, znamko, model, moč motorja, vrsto menjalnika in vrsto goriva. 9 Pri normalizaciji smo morali vse te podatke sproti izpisovati in poskrbeti za zanesljiv sistem obvladovanja napak. Zlasti v začetni fazi razvoja namreč pogosto prihaja do napak, za njihovo učinkovito odpravljanje pa je potreben podroben izpis poteka izvajanja **[logging-practices]**.

Poleg razlik v strukturi se posamezni viri med seboj razlikujejo tudi v zapisu posameznih vrednosti. Cene so na primer zapisane z ločilom tisočic, ki jih je treba pred pretvorbo v število odstraniti; prevoženi kilometri so pri enem viru zapisani kot celo število, pri drugem pa kot niz z enoto (na primer „123.456 km“); datum prve registracije je pri enem viru zapisan v obliki mesec/leto, pri drugem pa leto/mesec. Podobno je bilo treba poenotiti kategorialne vrednosti, kot sta tip menjalnika in vrsta goriva, saj so viri te podatke podajali v različnih jezikih.

Največji izziv normalizacije predstavlja poimenovanje vozil, saj sta ista znamka ali model na različnih virih pogosto zapisana drugače (okrajšave, dodatne besede, jezikovne razlike – na primer nemško „3er“ ali „Klasse“/„class“). Zato smo zgradili sistem ujemanja, ki surovo ime najprej poskuša natančno ujeti s kanoničnim imenom v podatkovni bazi, nato preveri tabelo že znanih psevdonimov (vodeno posebej za vsak vir), šele nazadnje pa uporabi približno (angl. *fuzzy*) ujemanje nizov **[fuzzy-string-matching]**. Za približno ujemanje smo uporabili knjižnico **RapidFuzz**, pri čemer smo prag ujemanja nastavili na 90 % za znamke in 75 % za modele. Nižji prag za modele smo izbrali zato, ker so imena modelov krajša in bolj podvržena lažnim ujemanjem. Vsako uspešno približno ujemanje se samodejno shrani kot nov psevdonim za

dani vir, s čimer se sistem sproti „učí“. Tako se vsak naslednji pojav istega zapisa razreši že v fazi iskanja psevdonimov, s čimer se izognemo počasnejšemu približnemu ujemanju.

Izsek kode 6: Ujemanje znamke s kanoničnim in približnim (fuzzy) ujemanjem

```
1 def match_brand(raw: str, source: str):
2     normalized = normalize_text(raw)
3     brands = _get_all_brands()
4
5     for brand_id, canonical_name in brands:
6         if normalize_text(canonical_name) == normalized:
7             return brand_id, canonical_name
8
9     cached = _lookup_brand_alias(normalized, source)
10    if cached:
11        return cached
12
13    brand_names = [normalize_text(b[1]) for b in brands]
14    result = process.extractOne(normalized, brand_names,
15                               ↪ scorer=fuzz.WRatio)
16
17    if result is None or result[1] < 90:
18        print(f"[matching] No brand match for '{raw}', best score:
19              ↪ {result[1] if result else 0}")
20        return None
21
22    brand_id, canonical_name = brands[result[2]]
23    _store_brand_alias(normalized, brand_id, canonical_name, source)
24    return brand_id, canonical_name
```

Po zaključeni normalizaciji so vsi podatki o oglasih vozil poenoteni v enotno strukturo (znamka, model, letnik, prevožena kilometrina, cena itd.), kar je predpogoj za primerjavo posameznih oglasov. Na tej podlagi smo implementirali API /*valuate*, ki kot vhodne parametre sprejme znamko, model, letnik in prevoženo kilometrino vozila, za katero uporabnik želi oceno tržne vrednosti. V tej fazi smo se namenoma omejili zgolj na navedene štiri parametre, saj je bil primarni cilj dokazati, da algoritem deluje na konceptualni

ravni (angl. *proof of concept*), preden bi ga razširili na dodatne spremenljivke (npr. opremljenost, vrsto goriva, menjalnik, stanje vozila). Dodajanje večjega števila parametrov brez zadostne količine normaliziranih podatkov bi namreč vzorec primerljivih vozil dodatno skrčilo, zaradi česar bi jih hitro ostalo premalo za smiselno statistično oceno.

Po svoji zasnovi algoritem ustreza pristopu primerljivih prodaj (angl. *comparable sales approach* oz. *Comparative Market Analysis* – CMA), ki je uveljavljena metoda vrednotenja v nepremičninski in avtomobilski stroki [22]. Cene predmeta pri tem pristopu ne ocenjujemo na podlagi enega samega modela, temveč na podlagi množice podobnih, dejansko prodanih (oz. oglaševanih) primerkov, ki jim glede na stopnjo podobnosti pripišemo različno utež. V statističnem smislu gre za obliko metode k -najbližjih sosedov (angl. *k-nearest neighbors*, k -NN) [1], pri kateri namesto ostre meje „sosed / ni sosed“ uporabimo zvezno utežno funkcijo, ki pada proti nič, ko se primerjano vozilo bolj razlikuje od ocenjevanega.

4.4.2 Filtriranje kandidatov iz podatkovne baze

Prvi korak algoritma je poizvedba SQL, ki iz tabele `market.normalized_market_listing` izlušči kandidate za primerjavo. Znamko in model filtriramo strogo (po enakosti). Znamka in model sta kategorični spremenljivki, ki najmočnejše določata segment in bazno ceno vozila.

Numerični spremenljivki filtriramo z dovoljenim odstopanjem: pri kilometri znaša $\pm 40\,000$ km (konstanta `KILOMETERS_DIFFERENCE`), pri letniku pa ± 3 leta (konstanta `YEAR_DIFFERENCE`). Navedeni meji nista bili izbrani naključno, temveč predstavljata kompromis med natančnostjo in velikostjo vzorca. Ožje kot postavimo meji, bolj primerljiva so vrnjena vozila glede na ocenjevano vozilo, a hkrati manjši je vzorec, iz katerega algoritem izračuna oceno, kar poveča statistično negotovost. Gre za znano razmerje med pristranskostjo in varianco (angl. *bias-variance trade-off*) [20]. Ker je podatkovna baza normaliziranih podatkov v tej fazi razvoja še razmeroma majhna, smo se zavestno odločili za širši interval, da algoritem sploh pridobi dovolj

velik vzorec za smiselno oceno. Ob prihodnji rasti podatkovne baze bi bilo smiselno ta interval zožiti oziroma ga narediti dinamičnega (npr. odvisnega od števila zadetkov ožjega intervala), s čimer bi izboljšali natančnost ocene, ne da bi ta trpela zaradi premajhnega vzorca.

4.4.3 Točkovanje podobnosti posameznega vozila

Za vsako vozilo iz filtrirane množice izračunamo oceno podobnosti (angl. *score*) glede na ocenjevano vozilo. Uporabili smo dve delni oceni, eno za kilometrino in eno za letnik, ki ju nato združimo v skupno oceno:

Izsek kode 7: Izračun ocene podobnosti posameznega vozila

```
1 def get_scores_of_each_car(data_compare: dict, cars: list) -> list[tuple]:
2     results = []
3     for car in cars:
4         absolute_dif = abs(int(car["kilometers"]) -
5         ↪ int(data_compare["kilometers"]))
6         kilometers_score = max(0, 100 * (1 - absolute_dif /
7         ↪ KILOMETERS_DIFFERENCE))
8
9         absolute_dif = abs(int(car["registration_year"]) -
10        ↪ int(data_compare["year"]))
11        registration_year_score = max(0, 100 * (1 - absolute_dif /
12        ↪ YEAR_DIFFERENCE))
13
14        total_score = kilometers_score * 0.6 + registration_year_score *
15        ↪ 0.4
16        results.append((float(car["price"]), total_score))
17    return results
```

Obe delni oceni sta zasnovani po istem načelu: vozilo, ki je identično ocenjevanemu, prejme največjih 100 točk, vozilo na robu dovoljenega intervala (natanko 40 000 km ali 3 leta narazen) pa 0 točk. Med tema skrajnostma ocena linearno pada. Funkcija `max(0, ...)` zagotavlja, da ocena nikoli ne postane negativna. Ker vozila zunaj dovoljenega intervala zaradi filtriranja v poizvedbi SQL sploh ne pridejo do tega koraka, gre pri tem predvsem za

dodatno varnostno mejo.

Za linearno padajočo funkcijo (namesto eksponentne ali Gaussove) smo se odločili iz dveh razlogov. Prvič, zaradi enostavnosti in razložljivosti: linearna funkcija je najlažje razumljiva in preverljiva, kar je pri diplomskem delu, kjer je cilj dokazati delujoč koncept, pomembnejše od marginalnega izboljšanja natančnosti. Drugič, zaradi enostavnega umerjanja: z eno samo konstanto natančno določimo, kje ocena doseže ničlo.

Skupno oceno podobnosti izračunamo kot uteženo vsoto obeh delnih ocen, pri čemer kilometrini pripišemo višjo utež (60 %) kot letniku (40 %). Vozili istega letnika se lahko močno razlikujeta po dejanski obrabi, če je eno prevozilo 50 000 km, drugo pa 200 000 km, medtem ko vozili s podobno kilometrino, a nekoliko različnim letnikom ostajata precej primerljivi.

4.4.4 Izračun ocenjene cene z uteženim povprečjem

Ko vsa primerljiva vozila ovrednotimo z oceno podobnosti, končno ocenjeno ceno izračunamo kot uteženo povprečje njihovih cen, pri čemer je utež vsakega vozila prav njegova izračunana ocena podobnosti:

$$\text{estimated_price} = \frac{\sum_i \text{price}_i \cdot \text{score}_i}{\sum_i \text{score}_i} \quad (4.1)$$

Uporaba uteženega povprečja namesto navadnega aritmetičnega je ključna: če bi izračunali navadno povprečje cen vseh vozil znotraj intervala, bi vozilo, oddaljeno 39 000 km, prispevalo enako kot vozilo, oddaljeno le 500 km, kar je neustrezno, saj je slednje veliko boljši pokazatelj dejanske cene. Z uteževanjem po vrednosti `total_score` bolj podobna vozila sorazmerno bolj vplivajo na končno oceno, vozila na robu intervala pa prispevajo le malo, gre za standardno obliko uteženega povprečja (angl. *weighted mean*) [4], kjer vsota uteži v imenovalcu poskrbi za pravilno normalizacijo rezultata, tako da ta ostane v enotah cene ne glede na število vozil ali velikost njihovih ocen. Poleg ocenjene cene algoritem vrne tudi indeks zaupanja (angl. *confidence*), izračunan po enačbi:

$$\text{confidence} = \min\left(1.0, \frac{n}{20}\right), \quad (4.2)$$

pri čemer je n število vozil, najdenih znotraj filtriranih meja. Kar pove uporabniku, je to kako smo prepričani v ceno, ki smo mu jo posredovali. Če je število primerjalnih vozil veliko, je indeks zaupanja bližje 1, če pa je število primerjalnih vozil majhno, je indeks zaupanja bližje 0.

4.4.5 Povzetek utemeljitve pristopa

Algoritem cene torej ne poskuša napovedati z enim samim „črnoškatlastim“ modelom, temveč uporablja pregleden in razločljiv pristop primerljivih prodaj z uteženim povprečjem, pri katerem kategorični spremenljivki (znamka, model) določata strogo mejo primerljivosti, numerični spremenljivki (kilometrina, letnik) določata mehko, zvezno padajočo utež znotraj te meje, končna cena je uteženo povprečje, ki daje večjo težo bolj podobnim vozilom, indeks zaupanja pa uporabnika pregledno opozori na zanesljivost posamezne ocene, glede na velikost vzorca. Ta pristop smo izbrali predvsem zato, ker je pri omejeni količini normaliziranih podatkov enostavnejši, bolj razločljiv in lažje preverljiv kot kompleksnejši statistični ali strojno naučeni modeli, hkrati pa metodološko temelji na uveljavljeni praksi vrednotenja rabljenih vozil in nepremičnin s primerjavo podobnih primerov na trgu. [22]

4.5 Infrastruktura in avtomatizacija

Poleg funkcionalne implementacije posameznih komponent je bil pomemben del razvoja tudi vzpostavitev infrastrukture, ki zagotavlja konsistentno delovanje sistema v različnih fazah njegovega življenjskega cikla. Od lokalnega razvoja, prek avtomatiziranega preverjanja kode, do namestitve v produkcijsko okolje. Namen tovrstne avtomatizacije je zmanjšati število ročnih posegov, ki so podvrženi napakam, in zagotoviti, da se aplikacija v vseh okoljih obnaša enako, ne glede na razlike v konfiguraciji posameznega razvijalčevega

računalnika ali strežnika [25]. Tak pristop temelji na načelu konsistentnosti okolij (angl. *environment parity*), po katerem naj bodo razvojno, testno in produkcijsko okolje čim bolj podobna, s čimer zmanjšamo tveganje napak, ki se pojavijo šele ob namestitvi [51].

4.5.1 Kontejnerizacija razvojnega okolja

Za kontejnerizacijo razvojnega okolja smo uporabili orodje Docker Compose, ki omogoča enostavnejšo vzpostavitev več med seboj povezanih kontejnerjev, naštetih v poglavju 1.1. Orodje deluje tako, da v datoteki `docker-compose.yml` opišemo vse storitve, ki jih aplikacija potrebuje, in njihove medsebojne odvisnosti, kar omogoča zagon celotnega okolja z enim samim ukazom. To je še posebej uporabno pri razvoju in testiranju. Za lokalno preizkušanje pošiljanja elektronske pošte smo uporabili orodje MailHog, ki odhodno pošto prestreže in jo prikaže v spletnem vmesniku, namesto da bi jo dejansko poslal na pravi naslov. Za zaledni in evalvacijski sistem smo napisali ločeni datoteki `Dockerfile`, v katerih določimo, na katerem jeziku oziroma okolju sistem temelji ter katere ukaze naj kontejner ob zagonu izvede.

Ker se storitve med seboj zaganjajo z različnimi hitrostmi, smo za pravilno zaporedje zagona uporabili ključ `depends_on` v kombinaciji z definicijami `healthcheck`. Brez tega bi se zaledni sistem lahko poskusil povezati s podatkovno bazo, še preden bi ta dejansko sprejemala povezave, kar bi ob zagonu okolja povzročilo napako. Storitve, od katerih so odvisne druge storitve, zato vsebujejo ukaz za preverjanje pripravljenosti (`pg_isready` pri sistemu PostgreSQL oziroma `redis-cli ping` pri sistemu Redis), odvisne storitve pa se zaženejo šele, ko je preverjanje uspešno.

Za shranjevanje podatkov smo uporabili poimenovane nosilce podatkov (angl. *named volumes*): eden skrbi za trajnost podatkovne baze med ponovnimi zagoni kontejnerjev, drugi pa predpomni mapo `node_modules` znotraj zalednega kontejnerja, saj bi jo sicer prepisala prevezava map z gostitelja. Za samo razvojno izkušnjo, natančneje za samodejno osveževanje ob spremembah (angl. *hot reload*), uporabljamo prevezavo map (angl. *bind mount*) iz

gostiteljevega datotečnega sistema v kontejner, tako da se spremembe v izvorni kodi takoj odražajo v delovanju aplikacije, brez ponovne gradnje slike. Odjemalskega sistema v lokalno razvojno okolje namenoma nismo vključili, saj njegova vključitev zaradi nenehne ponovne gradnje slike ob vsaki spremembi bistveno upočasni razvoj, zato ga med razvojem poganjamo neposredno na gostitelju.

Za produkcijsko okolje uporabljamo ločeno datoteko `docker-compose.prod.yml`, ki namesto lokalne gradnje slik uporablja vnaprej zgrajene slike iz registra vsebnikov (angl. *container registry*), storitve poveže v skupno omrežje ter doda poimenovan nosilec podatkov za shranjevanje naloženih datotek.

4.5.2 Neprekinjena integracija z orodjem GitHub Actions

Neprekinjena integracija (angl. *continuous integration*, CI) je praksa, pri kateri se ob vsaki spremembi kode samodejno izvedejo preverjanja (testi, gradnja, statična analiza kode), s čimer napake odkrijemo čim prej, še preden se zlijejo v glavno vejo ali dosežejo druge razvijalce [16]. Namesto da bi vsak razvijalec teste poganjal ročno pred združitvijo sprememb, jih namesto njega izvede strežnik, in sicer konsistentno ter vsakič na enak način.

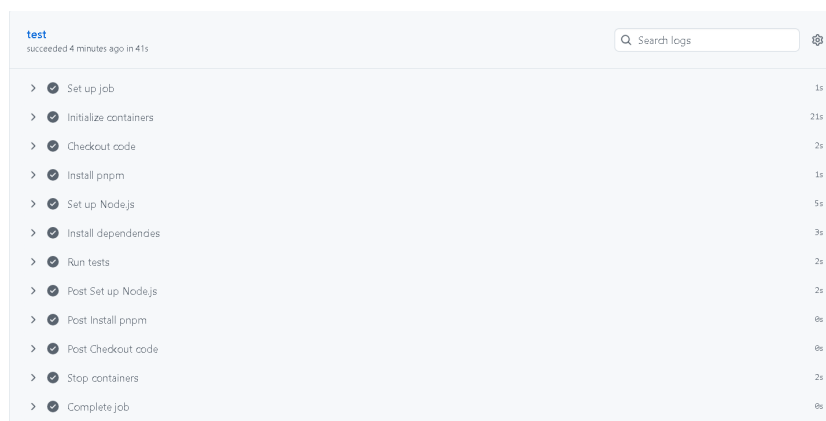
Za implementacijo neprekinjene integracije smo uporabili orodje GitHub Actions, GitHubov sistem za avtomatizacijo poteka dela. Njegovo delovanje temelji na naslednjih pojmi:

- **Potek dela** (angl. *workflow*) je datoteka `.yml` v mapi `.github/workflows/`, ki določa, kdaj naj se nekaj izvede (npr. ob dogodku `pull_request` ali `push`) in katera opravila naj se ob tem zaženejo.
- **Opravilo** (angl. *job*) se izvede na svežem virtualnem strežniku (npr. `ubuntu-latest`), ki ga GitHub ob zagonu ustvari in po zaključku izbriše.
- **Koraki** (angl. *steps*) znotraj opravila so zaporedni ukazi. To so lahko

neposredni lupinski ukazi (npr. `run: pnpm test`) ali vnaprej pripravljeni in ponovno uporabni gradniki (angl. *actions*), ki jih razvije GitHub ali skupnost. Tako npr. gradnik `actions/checkout` prenese izvirno kodo, gradnik `pnpm/action-setup` pa namesti upravitelja paketov pnpm. Namesto lastnoročno napisane skripte za namestitev okolja Node, predpomnjenje odvisnosti in nastavitve spremenljivke `PATH` tako uporabimo že obstoječi gradnik.

- **Storitve** (angl. *services*) so pomožni kontejnerji, ki tečejo vzporedno z opravilom. Uporabni so, kadar testi potrebujejo pravo podatkovno bazo namesto njene simulacije (angl. *mock*).

Potek dela, opredeljen v datoteki `ci.yml`, smo pripravili tako, da ob vsakem novem zahtevku za združitev (angl. *pull request*) izvede vse teste zalednega sistema in preveri, ali se uspešno izvedejo. Potek dela v datoteki `docker-publish.yml` je zasnovan še korak dlje: ob zahtevku za združitev zgolj preveri, ali se slika Docker uspešno zgradi, ob združitvi v glavno vejo pa sliko dejansko zgradi, digitalno podpiše in objavi. Ta korak že sodi na področje neprekinjene dostave (angl. *continuous delivery*, CD), torej samodejne distribucije zgrajenega artefakta [21].



Slika 4.4: Samodejno izvajanje testov nad vejo v okolju GitHub Actions

4.5.3 Namestitev in gostovanje

Za gostovanje smo izbrali eno izmed cenovno najugodnejših možnosti, in sicer najeti navidezni zasebni strežnik (angl. *virtual private server*, VPS) pri ponudniku Hetzner. Strežnik razpolaga s 4 GB delovnega pomnilnika in dvema procesorskima jedroma, kar zadostuje za zagon vseh treh storitev. Po prijavi v strežnik smo najprej posodobili operacijski sistem na najnovejšo različico, nato pa namestili orodji Docker in Docker Compose.

Za upravljanje aplikacije smo uporabili orodje Dokploy, ki omogoča enostavno uporabo datotek `docker-compose` v produkcijskem okolju in hkrati močno poenostavi postopek posodabljanja aplikacije. V njem lahko nastavimo tudi opravila, ki se izvajajo periodično, podobno kot storitev `cron`. Na strežnik je treba namestiti povezovalno komponento med orodjema Dokploy in Docker, nato pa Dokploy sam poskrbi za gradnjo aplikacije: iz repozitorija GitHub zajame najnovejšo različico kode in zgradi pripadajočo sliko neposredno na strežniku. Tak postopek omogoča, da se nova sprememba v produkcijskem okolju odrazi v nekaj minutah.

Dokploy je odprtokodno in brezplačno orodje, najem strežnika pri ponudniku Hetzner pa predstavlja nizek mesečni strošek.

Poglavje 5

Sklep

Cilj diplomskega dela je bil razviti spletno aplikacijo, ki uporabnikom omogoča objavo in pregled oglasov rabljenih avtomobilov, ter vanjo vgraditi sistem za samodejno ocenjevanje tržne vrednosti vozila na podlagi primerljivih vozil na trgu. Oba cilja sta bila dosežena: razvili smo delujočo spletno aplikacijo z ločeno odjemalsko, zaledno in evalvacijsko komponento (poglavje 3), zasnovali pa smo tudi cevovod za zajem, normalizacijo in primerjavo tržnih podatkov ter algoritem za oceno cene, ki temelji na pristopu primerljivih prodaj z uteženim povprečjem (razdelka 4.4.2 in 4.4.4).

Hipoteza diplomskega dela je bila, da je mogoče uporabniku na podlagi zbranih tržnih podatkov in primerjave s podobnimi vozili ponuditi uporabno in dovolj natančno oceno tržne vrednosti. Implementirani algoritem to hipotezo potrjuje na konceptualni ravni (angl. *proof of concept*). Sistem na podlagi znamke, modela, letnika in prevoženih kilometrov vrne oceno cene skupaj z indeksom zaupanja, ki uporabnika pregledno opozori na zanesljivost posamezne ocene glede na velikost vzorca primerljivih vozil (razdelek 4.4.4). Med izdelavo dela smo prepoznali več možnosti za nadaljnji razvoj, ki jih navajamo v nadaljevanju.

- **Razširitev nabora vhodnih spremenljivk.** Ko bo na voljo dovolj normaliziranih podatkov, bi bilo smiselno v algoritem vključiti dodatne parametre, kot so oprema, vrsta goriva, tip menjalnika in stanje vozila,

kar bi izboljšalo natančnost ocene.

- **Dinamično prilagajanje intervalov primerljivosti.** Namesto fiksnih mej bi lahko intervale kilometrine in letnika prilagajali sproti glede na gostoto podatkov v posameznem segmentu (na primer širši interval pri redkih modelih in ožji pri pogostih), s čimer bi omilili razmerje med pristranskostjo in varianco, opisano v razdelku 4.4.2.
- **Uvedba metod strojnega učenja.** Ko bo obseg lastnih, normaliziranih podatkov zadosten, bi lahko sedanji, razložljivi pristop primerljivih prodaj dopolnili ali nadomestili z modeli strojnega učenja (na primer regresijskimi modeli, naključnimi gozdovi ali nevronskimi mrežami), ki bi lahko zajeli bolj kompleksne, nelinearne odvisnosti med lastnostmi vozila in ceno [19].
- **Zmanjšanje odvisnosti od zunanjih virov.** Z rastjo števila uporabnikov in oglasov, objavljenih neposredno na platformi, bomo lahko delež zunanjih podatkov postopno zmanjševali in ocene vse bolj gradili na podatkih, zbranih znotraj same aplikacije, kar bo izboljšalo tako natančnost kot vzdržnost sistema, saj se s tem zmanjša odvisnost od krhkih postopkov zajema s spletnih strani.
- **Postopna migracija zalednega sistema na TypeScript.** Uvedba statičnega preverjanja tipov bi dolgoročno izboljšala zanesljivost in vzdržljivost zalednega dela aplikacije [17].
- **Nadgradnja infrastrukture in spremljanja delovanja.** Smiselna nadaljnja koraka sta uvedba centraliziranega beleženja dogodkov in spremljanja delovanja sistema (angl. *monitoring*) v produkcijskem okolju, kar bi olajšalo odkrivanje napak in analizo obremenitve sistema, ter razširitev nabora avtomatiziranih testov na evalvacijski in odjemalski del aplikacije, ki v tem trenutku nista pokrita enako temeljito kot zaledni sistem, za katerega teste ob vsakem zahtevku za združitev že izvaja orodje GitHub Actions.

Skupno gledano diplomsko delo dokazuje, da je zastavljeni koncept spletne aplikacije za prodajo vozil z vgrajenim, razločljivim sistemom za ocenjevanje tržne vrednosti izvedljiv in tehnično smiseln. Predstavlja trdno osnovo, na kateri je mogoče v prihodnosti graditi natančnejši, bolj podatkovno bogat in temeljiteje evalviran sistem.

Literatura

Članki v revijah

- [1] N. S. Altman. „An Introduction to Kernel and Nearest-Neighbor Non-parametric Regression“. V: *The American Statistician* 46.3 (1992), str. 175–185. DOI: 10.1080/00031305.1992.10475879.
- [8] Irfan Darmawan in sod. „Evaluating Web Scraping Performance Using XPath, CSS Selector, Regular Expression, and HTML DOM With Multiprocessing Technical Applications“. V: *JOIV: International Journal on Informatics Visualization* (2022). DOI: 10.30630/joiv.6.4.1525.
- [23] Marijn Janssen, Yannis Charalabidis in Anneke Zuiderwijk. „Benefits, Adoption Barriers and Myths of Open Data and Open Government“. V: *Information Systems Management* 29.4 (2012), str. 258–268. DOI: 10.1080/10580530.2012.716740.
- [27] Alberto H. F. Laender in sod. „A Brief Survey of Web Data Extraction Tools“. V: *ACM SIGMOD Record* 31.2 (2002), str. 84–93. DOI: 10.1145/565117.565137.
- [35] Christopher Olston in Marc Najork. „Web Crawling“. V: *Foundations and Trends in Information Retrieval* 4.3 (2010), str. 175–246. DOI: 10.1561/15000000017.
- [37] David L. Parnas. „On the Criteria To Be Used in Decomposing Systems into Modules“. V: *Communications of the ACM* 15.12 (1972), str. 1053–

1058. DOI: 10.1145/361598.361623. URL: <https://doi.org/10.1145/361598.361623>.

Članki v zbornikih konferenc

- [17] Zheng Gao, Christian Bird in Earl T. Barr. „To Type or Not to Type: Quantifying Detectable Bugs in JavaScript“. V: *Proceedings of the 39th International Conference on Software Engineering (ICSE)*. IEEE Press, 2017, str. 758–769. DOI: 10.1109/ICSE.2017.75.
- [19] Enis Gegic in sod. „Car Price Prediction using Machine Learning Techniques“. V: *TEM Journal*. Zv. 8. 1. 2019, str. 113–118. DOI: 10.18421/TEM81-16.
- [38] Niels Provos in David Mazières. „A Future-Adaptable Password Scheme“. V: *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*. 1999, str. 81–91.

Ostala literatura

- [2] Axios Contributors. *Axios – Promise based HTTP client for the browser and node.js*. <https://axios.rest>. Uradna dokumentacija knjižnice. 2024.
- [3] Michael Bayer. *Alembic Documentation*. Dostopano: 2026-07-26. 2024. URL: <https://alembic.sqlalchemy.org/>.
- [4] Philip R. Bevington in D. Keith Robinson. *Data Reduction and Error Analysis for the Physical Sciences*. 3. izd. New York, NY: McGraw-Hill, 2003. ISBN: 9780072472271.
- [5] BullMQ Contributors. *BullMQ Documentation*. Dostopano: junij 2024. 2024. URL: <https://docs.bullmq.io/>.
- [6] Express.js Contributors. *Express.js Documentation*. Dostopano: junij 2024. 2024. URL: <https://expressjs.com/>.
- [7] Sequelize Contributors. *Sequelize Documentation*. Dostopano: junij 2024. 2024. URL: <https://sequelize.org/>.
- [9] Ali Darvish Darab. *RESTful API – CS421 Course Notes*. Dostopano: 28. 6. 2026. 2020. URL: https://darvishdarab.github.io/cs421_f20/docs/readings/restful/api/.
- [10] Edsger W. Dijkstra. „On the Role of Scientific Thought“. V: *Selected Writings on Computing: A Personal Perspective*. New York, NY: Springer-Verlag, 1982, str. 60–66. DOI: 10.1007/978-1-4612-5695-3_12.
- [11] Nicola Dragoni in sod. „Microservices: Yesterday, Today, and Tomorrow“. V: *Present and Ulterior Software Engineering*. Springer International Publishing, 2017, str. 195–216. ISBN: 9783319674254. DOI: 10.1007/978-3-319-67425-4_12.
- [12] Ramez Elmasri in Shamkant B. Navathe. *Fundamentals of Database Systems*. 7. izd. Pearson, 2015. ISBN: 9780133970777.

- [13] Eurostat. *Eurostat – European Statistics*. Dostopano: junij 2025. 2024. URL: <https://ec.europa.eu/eurostat>.
- [14] Roy Thomas Fielding. „Architectural Styles and the Design of Network-based Software Architectures“. Doktorska disertacija. University of California, Irvine, 2000. URL: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [15] David Flanagan. *JavaScript: The Definitive Guide*. 7. izd. O'Reilly Media, 2020. ISBN: 9781491952023.
- [16] Martin Fowler. *Continuous Integration*. <https://martinfowler.com/articles/continuousIntegration.html>. Dostopano: 2026-08-01. 2006.
- [18] Hector Garcia-Molina, Jeffrey D. Ullman in Jennifer Widom. *Database Systems: The Complete Book*. 2. izd. Pearson, 2008, str. 587–625. ISBN: 9780131873254.
- [20] Trevor Hastie, Robert Tibshirani in Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2. izd. New York, NY: Springer, 2009. ISBN: 9780387848570. DOI: 10.1007/978-0-387-84858-7.
- [21] Jez Humble in David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA: Addison-Wesley, 2010. ISBN: 9780321601919.
- [22] Appraisal Institute. *The Appraisal of Real Estate*. 15. izd. Chicago, IL: Appraisal Institute, 2020. ISBN: 9781935328780.
- [24] Michael B. Jones, John Bradley in Nat Sakimura. *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force (IETF). Maj 2015. DOI: 10.17487/RFC7519. URL: <https://www.rfc-editor.org/rfc/rfc7519>.
- [25] Gene Kim in sod. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. 2. izd. Portland, OR: IT Revolution Press, 2021. ISBN: 9781950508402.

- [26] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. 1. izd. O'Reilly Media, 2017. ISBN: 978-1449373320.
- [28] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. ISBN: 9780134494166.
- [29] Meta Platforms, Inc. *React – The library for web and native user interfaces*. <https://react.dev>. Uradna dokumentacija. 2024.
- [30] Meta Platforms, Inc. *React Documentation: Describing the UI*. Dostopano: 21. 6. 2026. 2024. URL: <https://react.dev/learn/describing-the-ui>.
- [31] Microsoft. *TypeScript: JavaScript With Syntax For Types*. Dostopano: junij 2025. 2024. URL: <https://www.typescriptlang.org/>.
- [32] Priyanshu Mishra. *Task Queues and Background Jobs: A Backend Developer's Guide*. Dostopano: 11. 7. 2026. 2026. URL: <https://medium.com/@21je0710/task-queues-and-background-jobs-a-backend-developers-guide-470d22d52666>.
- [33] Mozilla Developer Network. *JavaScript modules*. <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>. Tehnična dokumentacija. 2024.
- [34] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2015. ISBN: 9781491950357.
- [36] Orgesleka. *Used Cars Database – eBay Kleinanzeigen*. Dostopano: junij 2025. 2016. URL: <https://www.kaggle.com/datasets/orgesleka/used-cars-database>.
- [39] Sebastián Ramírez. *FastAPI Documentation*. Dostopano: 2026-07-26. 2024. URL: <https://fastapi.tiangolo.com/>.
- [40] Amir Rawdat. *Rate Limiting with NGINX and NGINX Plus*. Dostopano: 21. 6. 2026. 2017. URL: <https://www.nginx.com/blog/rate-limiting-nginx/>.

-
- [41] Redis Ltd. *Redis Documentation*. Dostopano: junij 2024. 2024. URL: <https://redis.io/docs/latest/>.
- [42] Leonard Richardson in Sam Ruby. *RESTful Web Services*. O'Reilly Media, 2007. ISBN: 9780596529260.
- [43] Yaron Sheffer, Dick Hardt in Michael B. Jones. *JSON Web Token Best Current Practices*. Dostopano: junij 2024. 2020. DOI: 10.17487/RFC8725. URL: <https://www.rfc-editor.org/rfc/rfc8725>.
- [44] Statistični urad Republike Slovenije. *Statistični urad Republike Slovenije – SURS*. Dostopano: junij 2025. 2024. URL: <https://www.stat.si/StatWeb/>.
- [45] The Linux man-pages project. *crontab(5) — Linux manual page*. Dostopano: 2026-07-26. 2023. URL: <https://man7.org/linux/man-pages/man5/crontab.5.html>.
- [46] The PostgreSQL Global Development Group. *About PostgreSQL*. Dostopano: 21. 6. 2026. 2025. URL: <https://www.postgresql.org/about/>.
- [47] The PostgreSQL Global Development Group. *PostgreSQL Documentation: What Is PostgreSQL?* Dostopano: 21. 6. 2026. 2025. URL: <https://www.postgresql.org/docs/current/intro-what-is.html>.
- [48] Vite Contributors. *Vite – Next Generation Frontend Tooling*. <https://vite.dev>. Uradna dokumentacija orodja. 2024.
- [49] Evan Wallace. *esbuild – An extremely fast bundler for the web*. <https://esbuild.github.io>. Uradna dokumentacija orodja. 2024.
- [50] WHATWG. *HTML Living Standard – Web Storage*. <https://html.spec.whatwg.org/multipage/webstorage.html>. Uradna specifikacija. 2024.
- [51] Adam Wiggins. *The Twelve-Factor App*. Dostopano: 2026-08-01. 2017. URL: <https://12factor.net/>.

Dodatek

Koda: Definicija modela User

Izsek kode 8: Definicija modela `User` z ORM knjižnico Sequelize.

```
1 import { DataTypes } from "sequelize";
2 import sequelize from "../config/db.js";
3
4 const User = sequelize.define(
5   "User",
6   {
7     id: {
8       type: DataTypes.INTEGER,
9       autoIncrement: true,
10      primaryKey: true,
11    },
12    firstName: {
13      type: DataTypes.STRING,
14      allowNull: false,
15    },
16    lastName: {
17      type: DataTypes.STRING,
18    },
19    email: {
20      type: DataTypes.STRING,
21      allowNull: false,
22      unique: true,
23      validate: {
24        isEmail: true,
25      },
26    },
27    telephone: {
28      type: DataTypes.STRING,
```

```
29     allowNull: false,
30   },
31   password: {
32     type: DataTypes.STRING,
33     allowNull: false,
34   },
35   soldCars: {
36     type: DataTypes.INTEGER,
37     defaultValue: 0,
38   },
39 },
40 { timestamps: true },
41 );
42
43 export default User;
```

Izsek kode 9: Primer normalizacije podatkov

```
1 def extract_data_from_raw_payload(raw
2   valid = True
3   data = {}
4
5   raw_brand = raw_payload.get("brand", None)
6   raw_model = raw_payload.get("model", None)
7
8   if not raw_brand:
9     valid = False
10  if not raw_model:
11    valid = False
12
13  if valid:
14    brand_match = match_brand(_preprocess(raw_brand), SOURCE)
15    if brand_match is None:
16      print(f"Could not match brand '{raw_brand}' for raw_id:
17        ↳ {id}")
18      valid = False
19    else:
20      brand_id, brand_canonical = brand_match
21      data["brand_name"] = brand
22
23      model_match = match_model
24      _preprocess(raw_model), brand_id, brand_canonical, SOURCE
```



```
24         )
25         if model_match is None:
26             print(
27                 f"Could not match model '{raw_model}' (brand:
                ↪ '{brand_canonical}') for raw_id: {id}"
28             )
29             valid = False
30         else:
31             _, model_canonical = model_match
32             data["model_name"] =
```